URBANDHAGAA

Order Management System

System Design & Implementation Documentation

Purpose

A forward-looking blueprint for a corporate merchandise and gifting business. It defines the operating model, domain modules, core data entities, APIs, controls, security, testing strategy, and phased implementation using Python, FastAPI, PostgreSQL, SQLAlchemy, Alembic, Pydantic, and React.

Design objective

Support today's order processing while creating a stable foundation for future inventory, vendor collaboration, financial integrations, automation, analytics, and higher order volume—without prematurely adopting microservices.

Document status

Baseline system design. Business defaults are proposed where decisions were not explicitly finalized. Tax, statutory invoicing, and accounting behavior must be validated with a qualified Indian accounting professional before production use.

Document contents

1. Executive summary and design principles
2. Scope, users, and business capabilities
3. Finalized decisions and recommended defaults
4. End-to-end workflows and state machines
5. Functional requirements by module
6. Data model and entity relationships
7. API design conventions and endpoint catalogue
8. Order ID generation and transaction integrity
9. Pricing, GST, payments, and invoicing controls
10. Procurement, production, quality, and shipping
11. Security, auditability, and privacy
12. Non-functional requirements and operations
13. Testing and acceptance criteria
14. Phased roadmap and release gates
15. Risks, open configuration points, and glossary

1. Executive Summary and Design Principles

UrbanDhagaa needs an order management system tailored to made-to-order corporate merchandise, branded apparel, employee welcome kits, and corporate gifting. Orders can contain multiple products and variants, require artwork approval, use several suppliers, be produced in batches, ship in partial consignments, and be paid in installments.

The OMS should provide one source of truth for the order and connect sales, design, procurement, production, quality, dispatch, and accounts. It must distinguish commercial commitment from operational progress and financial settlement.

Architecture decision

Use a modular monolith: one deployable FastAPI application with clear domain modules, a PostgreSQL database, background workers for durable jobs, and private object storage for files. This is simpler to operate than microservices and can be decomposed later if genuine scale or team boundaries require it.

Core principles

• Single source of truth: every business record has a canonical owner and stable identifier.

• Immutable issued references: never reuse or silently rewrite issued Order IDs, invoices, payment records, or approved design versions.

• Separate concerns: order, design, payment, production, and delivery statuses are independent.

• Snapshot commercial facts: preserve agreed item descriptions, quantities, prices, discounts, and tax treatment at confirmation.

• Traceability: record who did what, when, and why for material actions.

• Configurable rules: payment gates, tax configuration, approval limits, and notifications must not be buried as hard-coded assumptions.

• Incremental delivery: ship a reliable core before adding warehouse, vendor portal, or advanced analytics features.

2. Scope, Users, and Business Capabilities

In-scope capabilities

• Customer/company records, contacts, billing and shipping addresses.

• Product catalogue, SKU/variants, customization attributes, size and colour breakdowns.

• Order creation, unique Order ID, line items, deadlines, ownership, notes, status history.

• Quotation versions, customer acceptance, commercial terms, discounts, configurable tax calculations.

• Payment receipts, allocations, refunds/adjustments, outstanding balance and reconciliation.

• Artwork files, mockups, versioning, design/sample approvals.

• Vendor directory, purchase orders, supplier receipts, supplier invoices and costs.

• Production batches, quantities, rework, quality inspections and accepted output.

• Multiple shipments per order, tracking, partial delivery, proof of delivery and claims.

• Customer invoices/credit notes records, reporting, notifications, audit logs and role-based access.

Out of scope for the first release

• Full accounting/general ledger replacement.

• Complex multi-warehouse optimization and advanced forecasting.

• Microservices, event streaming infrastructure, and Kubernetes unless later justified.

• Automated statutory tax interpretation without accountant-approved rules.

• Public vendor access portal before internal workflows are stable.

Roles

Role	Typical permissions
ADMIN	Manage users, configuration, overrides, and system administration.
SALES	Manage customers, leads if enabled, quotations, order drafts, customer communications.
OPERATIONS	Manage confirmed order execution, schedules, production coordination, dispatch readiness.
DESIGNER	Access assigned design jobs, upload revisions, record artwork/mockup status.
PROCUREMENT	Manage vendors, supplier POs, receipts, supplier costs.
ACCOUNTS	Manage payment records, allocations, invoices, refunds/adjustments and receivables.
READ_ONLY	View permitted records and reports without modifying them.
VENDOR (future)	Access only assigned vendor work, with restricted fields and no cross-customer access.

3. Finalized Decisions and Recommended Defaults

The user has selected the Python/FastAPI direction and accepted the general order ID design. The following defaults complete the baseline specification; they should be configurable where practical.

Decision	Baseline recommendation	Why / change path
Architecture	Modular monolith	Simple to build and operate; modules can be separated later.
Canonical Order ID	UD-YYYY-NNNNN	Stable human-readable ID; category remains a separate field.
Sequence	One shared annual sequence	One order register across all categories; generated atomically in PostgreSQL.
Year basis	Calendar year by default	Matches the accepted UD-YYYY-NNNNN pattern. Changing to Indian financial year later requires a deliberate numbering policy and migration plan.
Payment gate	Configurable per order/customer policy	Supports advance payments and approved credit terms.
Inventory	Procurement-first initially; design for hybrid later	Track supplier purchasing now; add stock ledger/reservations when stocked products justify it.
Vendor access	Internal team coordinates vendors initially	Future vendor portal can be added using scoped permissions.
Invoicing	OMS stores invoice records and supports invoice generation as a controlled module	Use accountant-approved templates and numbering/tax rules; integrate accounting software if needed.
Frontend	React	Web dashboard can support office and mobile browser workflows.
Background jobs	Durable queue/outbox for important jobs	Reliable retries for reminders and notifications.

These are design defaults, not claims that the user explicitly selected each option. Business configuration can be finalized before production rollout.

4. End-to-End Workflows and State Machines

Order lifecycle

Draft → Quoted → Confirmed → In Fulfilment → Partially Dispatched (if applicable) → Delivered → Closed. Cancellation is a separate controlled transition, not deletion. The exact states may be represented with enums or reference tables, but transition rules must be centralized in domain services.

Independent status dimensions

Dimension	Example values
Order	Draft, Quoted, Confirmed, In Fulfilment, Delivered, Cancelled, Closed
Design	Not Required, Pending, In Review, Changes Required, Approved
Payment	Unpaid, Partially Paid, Paid, Overdue, Partially Refunded, Refunded
Procurement	Not Required, Not Ordered, Ordered, Partially Received, Received
Production	Not Started, In Progress, Quality Check, Rework, Completed
Delivery	Not Dispatched, Partially Dispatched, In Transit, Delivered, Exception

Transition gates

Action	Required validations
Confirm order	Customer and items exist; quantities, specifications, agreed prices, delivery date and terms are saved; accepted quote or documented approval exists.
Release production	Required artwork/sample approvals are recorded; supplier/material plan exists; configured advance/credit rule is satisfied or an authorized exception is recorded.
Complete production	Accepted, rejected, and rework quantities reconcile to planned quantity.
Dispatch	Shipment lines reference order items; quantities do not exceed eligible undelivered quantities; required QC is passed.
Mark delivered	Proof of delivery or authorized exception is recorded; partial delivery remains distinguishable.
Close order	Fulfilment and financial closure rules pass, or documented exceptions are approved.
Cancel	Reason is mandatory; work-in-progress, supplier commitments, customer refund/credit, and already shipped quantities are reconciled.

Do not hard-code full payment as a universal dispatch requirement. Dispatch eligibility should follow the agreed credit terms and configured policy.

5. Functional Requirements by Module

5.1 Customers and contacts

• Create a customer/company record with unique Customer ID, legal/display name, type, status, and account owner.

• Support multiple contacts and multiple billing/shipping addresses per customer.

• Store GSTIN and tax details only where applicable; validate format and preserve change history.

• Support customer notes, communication preferences, payment terms, and optional credit limits.

• Prevent accidental duplicate customers using search and duplicate warnings; do not merge automatically without review.

5.2 Product catalogue and variants

• Maintain SKU, name, category, description, active status, default specification, and default cost/price references.

• Support configurable attributes such as size, colour, GSM, material, capacity, print method, print position, packaging and personalization.

• Use structured variants where practical; use a size allocation table for bulk apparel quantities.

• Preserve order-item snapshots so catalogue changes do not rewrite confirmed orders.

• Support made-to-order products that are not held as stock.

5.3 Orders and order items

• Create orders with generated ID, customer, category, owner, source, priority, requested/committed delivery dates, customer PO reference, and notes.

• Support multiple line items, separate specifications, quantity, price, discount, tax treatment and customization per item.

• Record requested delivery date separately from committed delivery date.

• Support amendments through controlled revisions; record who approved scope, price, or deadline changes.

• Support cancellation with reason and a full event history; never delete confirmed orders as a normal workflow.

5.4 Quotations

• Generate quote numbers separately from Order IDs; support multiple versions and validity dates.

• Snapshot line descriptions, quantities, rates, discounts, charges, tax assumptions, delivery estimates and terms.

• Record sent, viewed (if tracked), accepted, rejected, expired, and superseded states.

• An accepted quotation can create or confirm an order only through a transaction-safe workflow.

• Do not silently edit a quote version already sent to the customer; create a new revision.

5.5 Pricing, taxes and payments

• Use Decimal in Python and NUMERIC in PostgreSQL for monetary values.

• Define calculation order and rounding policy for line discounts, order discounts, freight, packaging, taxable value and taxes.

• Keep payment transactions separate from payment allocations; support partial payments and one payment allocated across invoices/orders if business rules allow.

• Record payment method, transaction reference, received date, amount, currency, receipt evidence, and recorded-by user.

• Support failed/reversed transactions and refunds as explicit records, not destructive edits.

• Derive outstanding balances from invoices, credits, payments and allocations; reconcile periodically.

5.6 Design and approvals

• Store secure file metadata, file version, uploader, timestamp, checksum, and related order item.

• Record mockup/sample review state, approver, approval time, version approved and comments.

• Any material artwork change after approval creates a new version and may invalidate the previous approval.

• Production release checks whether approval is required for each item.

• Provide expiring, authorized download access rather than public permanent file URLs.

5.7 Vendors and procurement

• Maintain vendor records, contacts, capabilities, lead times, terms and approved status.

• Create supplier POs with separate PO numbers and multiple PO items; link each to the relevant order items where applicable.

• Track ordered, received, accepted, rejected and outstanding quantities.

• Record supplier invoices and actual costs; support partial receipts and multiple vendors per customer order.

• Do not treat a PO as received stock or an invoice as paid without the corresponding recorded event.

5.8 Production and quality control

• Create production batches linked to order items, suppliers/team, planned quantities and planned dates.

• Track milestones, material readiness, printing/branding, completion, accepted output, rejects and rework.

• Record QC checklist/result, inspector, time, defects, corrective action and reinspection result.

• Do not count rejected/rework quantities as accepted output until they pass QC.

• Allow production to be split into batches and track actual cost where available.

5.9 Shipment and delivery

• Support multiple shipments per order and multiple order items per shipment.

• Track packed/dispatch quantities, carton count, carrier, tracking/LR number, shipping cost, dates and proof of delivery.

• Validate that total shipped quantities cannot exceed eligible quantities, allowing only explicitly authorized exceptions.

• Track partial delivery, lost/damaged/short shipments, claims, replacements and returns.

• Keep delivered quantity separate from invoiced quantity and paid amount.

5.10 Invoices and accounting handoff

• Maintain invoice number, date, type, customer snapshot, linked order, line details, taxable value, taxes, total, due date and status.

• Use a separate invoice numbering policy from Order ID and validate statutory requirements before production use.

• Preserve issued invoices; corrections should use appropriate credit/debit note or cancellation/reissue workflows as applicable.

• Support partial billing where the business permits it, including reconciliation of invoice lines to order/shipment quantities.

• Provide export/integration boundaries for accounting software rather than pretending the OMS is a full general ledger.

5.11 Tasks, notifications and reports

• Create assigned tasks with due dates, priority, status, linked order and completion evidence where relevant.

• Use durable background jobs/outbox records for important email/WhatsApp notifications and retry failures.

• Provide dashboards for open orders, orders due soon, design approvals pending, procurement delays, production status, dispatches, overdue receivables and margins.

• Define each KPI precisely and include date ranges and filters.

• Keep communication logs with message type, recipient, status and related order where appropriate.

6. Data Model and Entity Relationships

Recommended logical entities

Entity group	Representative tables	Key responsibilities
Identity	users, roles, user_roles, sessions/refresh_tokens if used	Authentication, permissions, user lifecycle.
Customer	customers, customer_contacts, customer_addresses	Company identity and reusable contact/address records.
Catalogue	products, product_categories, product_variants, product_attribute_values	Reusable catalogue and variants.
Orders	orders, order_items, order_events, order_revisions	Order header, lines, immutable identity, transition history.
Quotes	quotations, quotation_items, quotation_versions/acceptances	Versioned commercial offers and acceptance.
Finance	payments, payment_allocations, invoices, invoice_items, credit_notes, refunds	Receipts, allocation, billing and reconciliation.
Design	design_assets, design_versions, approvals	Secure files, versions and approvals.
Procurement	vendors, purchase_orders, purchase_order_items, goods_receipts, receipt_items	Supplier commitments and received quantities.
Production	production_batches, production_events, quality_checks, rework_records	Batch progress, accepted/rejected quantities and QC.
Fulfilment	shipments, shipment_items, delivery_events, claims, returns	Partial shipments, proof of delivery and exceptions.
Operations	tasks, notifications, notification_attempts, outbox_events	Durable work and communication delivery.
Governance	audit_logs, order_counters, app_settings	Audit, sequence allocation and configurable policy.

Database constraints and conventions

• Use UUID primary keys or generated numeric primary keys internally; keep human-readable codes as separate unique columns.

• Add foreign keys and appropriate indexes for customer, status, dates, order code, SKU, vendor, invoice number and tracking number.

• Use UNIQUE constraints for Order ID, quotation number within its scope, invoice serial within its legal numbering scope, and idempotency keys.

• Use CHECK constraints for positive quantities, nonnegative sequence values, and valid amount boundaries where practical.

• Use NUMERIC for money; use timezone-aware timestamps in UTC and display them in the configured business timezone.

• Use explicit deletion policies. Financial and confirmed operational records should normally be archived/voided through controlled workflows, not cascaded away.

• Keep audit history append-only for application roles; restrict direct database access.

Core relationship notes

• One customer has many orders, contacts and addresses.

• One order has many items, quotation versions, payment allocations, invoices, batches, shipments and events.

• One shipment has many shipment items; each shipment item references an order item.

• One purchase order has many supplier line items and may receive goods in multiple receipts.

• One payment can be allocated to one or more receivables according to the chosen accounting policy.

• An order item may be produced in multiple batches and shipped across multiple shipments.

7. API Design Conventions and Endpoint Catalogue

Use a versioned REST API under /api/v1. Keep routers thin; business rules belong in service/domain functions, and database operations should be transaction-aware.

Domain	Representative endpoints
Customers	POST/GET /api/v1/customers; GET/PATCH /api/v1/customers/{id}; nested contacts/addresses
Products	POST/GET /api/v1/products; GET/PATCH /api/v1/products/{id}; variants
Orders	POST/GET /api/v1/orders; GET/PATCH /api/v1/orders/{id}; POST /{id}/confirm; POST /{id}/cancel
Quotations	POST /api/v1/orders/{id}/quotations; POST /api/v1/quotations/{id}/send; POST /{id}/accept
Design	POST /api/v1/orders/{id}/design-assets; POST /api/v1/design-assets/{id}/approve
Procurement	POST /api/v1/purchase-orders; POST /api/v1/purchase-orders/{id}/receipts
Production	POST /api/v1/order-items/{id}/batches; POST /api/v1/batches/{id}/quality-checks
Payments	POST /api/v1/payments; POST /api/v1/payments/{id}/allocations; POST /api/v1/payments/{id}/refunds
Invoices	POST /api/v1/orders/{id}/invoices; GET /api/v1/invoices; GET /api/v1/invoices/{id}
Shipments	POST /api/v1/orders/{id}/shipments; POST /api/v1/shipments/{id}/dispatch; POST /{id}/delivery-events
Reports	GET /api/v1/reports/order-pipeline; /receivables; /delivery-risk; /margins

API rules

• Use Pydantic request/response schemas; do not expose SQLAlchemy models directly.

• Use consistent pagination, sorting, filtering, and error response shapes.

• Return appropriate HTTP status codes and stable error codes.

• Use idempotency keys for order creation, payment recording, and other retry-prone write operations.

• Use explicit action endpoints for confirmation, cancellation, approval, dispatch, and closure.

• Document APIs using FastAPI's generated OpenAPI schema; protect production documentation if required by policy.

• Never trust user-supplied totals; recompute monetary totals on the server.

8. Order ID Generation and Transaction Integrity

Canonical Order ID

UD-2026-00001

Format: UD-YYYY-NNNNN. Use one shared sequence per calendar year by default. Category is a separate field (MER, GFT, WKT, MIX, or another configured category), not part of the canonical identity.

Generation requirements

• Use an order_counters table keyed by year or an equivalent database-backed sequence strategy.

• Increment the counter atomically in the same transaction that creates the order.

• Enforce a unique constraint on the generated Order ID.

• Use the business timezone to determine the applicable year.

• Never reuse a committed Order ID, including after cancellation.

• Do not generate identifiers using SELECT MAX(...) + 1, spreadsheet row numbers, or process-local counters.

• Define what happens if a sequence reaches its configured maximum; alert before exhaustion and expand the format deliberately.

Transaction boundary

A single transaction should allocate the sequence, create the order header, insert order items, save initial status/event records, and commit. If any required write fails, the transaction should roll back. External notifications should be triggered after commit through a durable outbox or queue, not before the order exists.

9. Pricing, GST, Payments, and Invoicing Controls

• Represent monetary values using Python Decimal and PostgreSQL NUMERIC with documented precision/scale.

• Define and test the order of operations for line discounts, order discounts, freight, packaging, other charges, taxable value, tax and rounding.

• Store the tax configuration and relevant tax snapshot used when the quotation/order/invoice was issued.

• Do not assume a single GST rate applies to all merchandise, services, charges, or transactions; validate treatment with an Indian tax professional.

• Separate quote number, Order ID, supplier PO number, shipment ID, payment reference and tax invoice serial.

• Preserve issued commercial/tax documents. Use controlled correction, credit-note, debit-note, cancellation or reissue workflows as applicable.

• Payment status should be derived from invoice balances and allocations under documented rules, not maintained as an independently editable truth.

• Support payment overage, underpayment, unapplied receipts, refunds and reconciliation exceptions explicitly.

• Maintain estimated and actual costs separately so gross margin reports disclose their completeness.

Suggested financial definitions

Metric	Definition
Order value	Confirmed commercial amount under the defined inclusion/exclusion policy.
Amount received	Successfully recorded customer receipts, subject to reversal/refund policy.
Outstanding receivable	Due invoice balances less allocated receipts and valid credits.
Estimated gross margin	Estimated net revenue less estimated direct costs.
Actual gross margin	Net revenue less recorded actual direct costs and relevant adjustments.
Vendor payable	Supplier invoice/receipt obligations less recorded supplier payments/credits, according to accounting policy.

This OMS design is not a substitute for statutory accounting advice. Invoice formats, numbering scopes, tax calculations, records, and retention must be checked against current Indian GST requirements before production.

10. Procurement, Production, Quality, and Shipping

Procurement

• Track supplier PO quantity separately from received, accepted, rejected, and outstanding quantity.

• Allow multiple suppliers and partial receipts for one customer order.

• Record actual supplier cost and freight allocation where possible.

• Do not count ordered goods as available inventory before a receipt/inspection event.

Production

• Use production batches linked to specific order items and quantities.

• Capture planned versus actual milestones and assigned owner/vendor.

• Track accepted output, rejected output, rework and QC results separately.

• Record approved artwork version against the production batch or item.

• Support exception handling when a committed delivery date is at risk.

Inventory evolution

Initial mode: procurement-first, suitable when most products are sourced or manufactured after an order is confirmed. Future hybrid mode should introduce a stock movement ledger, receipts, reservations, adjustments, returns, locations and stock availability calculations. Avoid storing a single manually edited quantity as the only inventory truth.

Shipping

• Support multiple shipments for one order and partial deliveries.

• Validate shipped quantities against eligible order-item quantities.

• Capture carrier, tracking/LR reference, cartons, freight, dispatch time, delivery proof and exception/claim records.

• Track replacements/returns as linked transactions rather than overwriting original shipment records.

11. Security, Auditability, and Privacy

Authentication and authorization

• Use a maintained authentication approach; hash passwords with a modern password-hashing algorithm if password login is used.

• Use short-lived access tokens and a deliberate refresh/revocation strategy if JWT is chosen.

• Enforce permissions on the server for every protected endpoint and record-level operation.

• Restrict access to financial actions, overrides, user administration, and vendor/customer data according to role.

• Use least-privilege application/database accounts and separate environments/secrets.

Audit and files

• Record actor, timestamp, action, entity, and meaningful before/after values for price, quantity, dates, status, payment, approval and permission changes.

• Do not log passwords, access tokens, payment credentials, or unnecessary personal data.

• Store files in private object storage; validate file type, size, and access; use short-lived signed URLs where suitable.

• Keep approved design versions immutable and record approval against the exact version.

• Define retention and deletion rules for customer records and business documents.

Operational security

• Use HTTPS, secure secret storage, dependency updates, input validation, rate limits and secure CORS configuration.

• Protect against unauthorized object access (IDOR), unsafe file uploads, injection, brute force and mass assignment.

• Back up the database and file metadata; test restoration regularly.

• Use structured logs, health checks, error monitoring and alerts without exposing sensitive values.

12. Non-Functional Requirements and Operations

Quality area	Baseline requirement
Correctness	Database constraints and transactions protect unique IDs, quantities, financial records and status changes.
Availability	Define a practical service target for the business; avoid promising uptime before hosting and operations are selected.
Performance	Paginate list APIs; index common filters; measure before optimizing. Establish target latency with representative data.
Scalability	Support increased order volume through indexed PostgreSQL, connection pooling, stateless API instances and background workers.
Recoverability	Document backup frequency, recovery point objective (RPO) and recovery time objective (RTO); test restore procedures.
Observability	Structured logs, request IDs, error monitoring, metrics, health checks and background-job failure alerts.
Maintainability	Typed Python, linting/formatting, migrations, module boundaries, code review and automated tests.
Portability	Environment-based configuration; no secrets in source control; reproducible dependency lock/constraints.
Accessibility/usability	Clear validation errors, search/filter, responsive layouts, keyboard-friendly forms and confirmation for destructive actions.
Data integrity	Audit trail, controlled amendments, idempotency, foreign keys and reconciliation reports.

Suggested project tooling

pytest, HTTPX for API tests, a formatter/linter such as Ruff, Alembic migrations, a reproducible dependency file, Docker Compose for local PostgreSQL if useful, and CI checks for tests, linting, migrations, and dependency security.

13. Testing and Acceptance Criteria

Test area	Acceptance criterion
Concurrent IDs	Concurrent order creation produces unique Order IDs with no duplicate committed references.
Rollback	A failed order transaction does not leave a partially created order or inconsistent counter state.
Idempotency	Repeating a write request with the same idempotency key does not duplicate the business operation.
Price snapshots	Catalogue edits do not change a confirmed order's historical specifications or price.
Quantity reconciliation	Size allocations, production output, QC accepted quantity, shipment quantities and delivery quantities reconcile.
Payment reconciliation	Partial payments, allocations, reversals and refunds produce the correct balance under the documented policy.
Authorization	Users cannot access or mutate unauthorized records by guessing identifiers or calling endpoints directly.
Approval gates	Required artwork/sample approval is enforced before production release.
Cancellation	Cancellation requires a reason and preserves all related events, commitments and financial adjustments.
Audit	Material changes record actor, time, action and relevant values.
Backup recovery	A backup is restored in a test environment and key records are reconciled.
Tax and documents	Approved test cases match accountant-reviewed expected calculations and invoice formats.

Testing layers

- Unit tests for pricing, state transitions, ID formatting, quantity calculations and policy checks.
- Integration tests using PostgreSQL for transactions, constraints, counter concurrency, and migrations.
- API tests for validation, authorization, idempotency, pagination and error behavior.
- End-to-end workflow tests from customer/order creation through quotation, production, dispatch and payment closure.
- Regression tests for financial calculation changes and migrations.

14. Phased Roadmap and Release Gates

Phase	Deliverables	Release gate
0. Discovery and foundation	Confirm policies, data dictionary, repository structure, config, database, migrations, CI and authentication approach.	Architecture decisions documented; environment reproducible.
1. Core OMS	Customers, products/variants, orders/items, ID generation, search/filter, status history, roles and audit.	Concurrent IDs, CRUD validation, permission and migration tests pass.
2. Commercials	Quotation versions, pricing/tax calculation module, payment records/allocations, invoice records/documents.	Accountant-reviewed examples and reconciliation tests pass.
3. Fulfilment	Design approval, vendor directory, purchase orders, receipts, production batches, QC, shipment and POD.	Partial quantity and approval-gate tests pass.
4. Automation and reports	Outbox/worker, reminders, dashboard KPIs, export, operational alerts.	Retries, duplicate handling and KPI definitions validated.
5. Scale and integrations	Hybrid inventory, vendor portal, accounting integration, customer portal, advanced analytics as justified.	Security review, load/recovery tests and business case approved.

Suggested implementation order

- Finalize business rules and data dictionary before generating migrations.
- Build the order counter and transaction tests early; they are foundational.
- Implement the simplest complete order workflow before adding every module.
- Deploy to staging with realistic sample orders and test users.
- Run acceptance tests and restore a backup before production cutover.
- Release features incrementally; do not enable optional modules until their controls are ready.

15. Risks, Configurations, and Glossary

Principal risks and mitigations

Risk	Mitigation
Scope creep	Prioritize core OMS and use phased releases.
Incorrect tax/invoice handling	Use configurable rules and accountant-reviewed statutory workflows.
Order ID collisions	Atomic database counter plus unique constraint and concurrency tests.
Untracked amendments	Version quotations/designs and record order change approvals.
Incorrect margins	Track estimates and actual costs separately; reconcile cost completeness.
Partial fulfilment confusion	Use shipment items and quantity reconciliation at order-item level.
Lost notifications	Durable outbox/queue, retries, delivery logs and dead-letter handling where supported.
Data loss	Automated backups, tested restoration and explicit RPO/RTO.
Unauthorized access	RBAC, object-level authorization, audit and least privilege.
Premature complexity	Modular monolith; add inventory portal/integrations only when justified.

Configuration points to finalize before go-live

- Business timezone and annual sequence reset policy.
- Category list and product-specific attribute rules.
- Payment/advance policies, credit approval limits and dispatch exceptions.
- GST configuration, invoice numbering and statutory document workflow.
- Approval thresholds for discounts, order changes, cancellations and refunds.
- File retention, backup frequency, RPO/RTO and access retention.
- Whether inventory is procurement-first or hybrid at launch.
- Notification channels, templates, customer consent and escalation timing.

Glossary

Term	Meaning
OMS	Order Management System.
SKU	Stock Keeping Unit; an identifier for a product or variant.
PO	Purchase Order issued to a supplier.
POD	Proof of Delivery.
QC	Quality Control.
RBAC	Role-Based Access Control.
Idempotency	Repeated equivalent requests do not create duplicate business effects.
Outbox pattern	Persist an event/message in the same database transaction as a business change, then deliver it asynchronously.
RPO / RTO	Recovery Point Objective / Recovery Time Objective.
Canonical Order ID	The stable human-readable order reference, separate from internal database primary key.

Final design recommendation

Build UrbanDhagaa's OMS as a modular FastAPI monolith with PostgreSQL as the source of truth. Keep the canonical Order ID immutable, use explicit state transitions and transaction-safe operations, preserve commercial snapshots, model partial payments and fulfilment, and enforce access and audit controls from the beginning. Design future inventory, vendor collaboration, and integrations as extensions—not as reasons to overcomplicate the first release.

This document is the baseline system design specification. It should be reviewed against the actual business process and validated by relevant finance/tax professionals before production use.